

SBA301 – SLOT 20

CHAPTER 14 – CREATE SINGLE PAGE APPLICATION

FULL-STACK SPA: REACT <-> SPRING BOOT • API URL • CORS • INTEGRATION

Tư duy trung tâm của Slot 20: Một SPA full-stack không chỉ là “frontend gọi được backend”. Sinh viên phải nhìn được toàn bộ browser boundary: React tạo request -> URL/origin được resolve -> browser áp dụng Same-Origin Policy/CORS -> Spring MVC xử lý request -> JSON quay về -> React cập nhật state và render. CORS là browser security contract, không phải lỗi Axios và không nên được “sửa” bằng cách mở wildcard vô điều kiện.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	20
Chủ đề	Chapter 14 – Full-Stack SPA Integration: React <-> Spring Boot, API URL, CORS
CLO chính	CLO4 – Integrate ReactJS frontend with Spring Boot RESTful Web Service
Kế thừa	Slot 18 REST/JPA boundary -> Slot 19 Lab/project guidance -> Slot 20 browser-to-backend integration
Case study xuyên suốt	Orchid Management / Lab 05
Môi trường khóa học	JDK 21 • Spring Boot 3.x • Maven • Spring Web • Spring Data JPA • SQL Server • React + Vite • Axios • Browser DevTools • Postman
Thời lượng gợi ý	135 phút trên lớp + Lab 05/project practice ngoài giờ
Learning flow	Prepare -> Understand -> Trace -> Implement -> Break -> Diagnose -> Verify -> Transfer

Scope lock: Slot 20 không dạy lại sâu React component cơ bản, REST Controller/JPA hay CRUD semantics đã học. Trọng tâm là integration boundary: API URL, origin, CORS, browser networking, environment configuration, request/response flow và cách chẩn đoán lỗi FE<->BE. Security/JWT chuyên sâu thuộc slot sau; deployment/CDN/reverse proxy được dùng ở mức production extension.

CÁCH SỬ DỤNG GIÁO TRÌNH

Pha học	Sinh viên làm gì?	Bằng chứng
Prepare	Chạy BE + FE baseline; xác nhận port, API và DB.	Baseline run + Git checkpoint
Understand	Phân biệt URL, origin, same-origin policy, CORS, preflight, client config.	Integration concept map
Trace	Theo dõi một GET/POST từ React qua browser Network tới Spring Boot và ngược lại.	Browser + backend execution trace
Implement	Tạo API client, base URL/env, tích hợp React <-> Spring Boot, cấu hình CORS.	Source + Network evidence
Break	Cố ý sai port/path/origin/header/CORS.	Failure evidence
Diagnose	Đọc browser console, Network tab, OPTIONS preflight, backend logs.	Debug note
Verify	Kiểm tra UI, HTTP status/body và DB hậu điều kiện.	Test matrix + DB evidence
Transfer	Thiết kế integration config cho staging/production hoặc module project.	Deployment/integration worksheet

AI rule: AI được dùng để khám phá network/execution rule, production integration pattern và technology update; mọi claim phải đối chiếu với source, Browser DevTools, backend log và official documentation. AI không thay thế evidence.

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC BUỔI NÀY

0.1. Kiến thức cần ôn lại

- React: component, props/state, useEffect ở mức đã học; Vite dev server; npm run dev.
- HTTP: method, URL, status, headers, JSON request/response; GET/POST/PUT/DELETE.
- Slot 18: Controller -> Service -> Repository -> JPA -> SQL Server; ResponseEntity và API contract.
- Khái niệm browser origin gồm scheme + host + port; localhost:5173 và localhost:8080 là hai origin khác nhau.
- Lab 05 baseline: Spring Boot REST API + SQL Server; React client dùng Axios và hiển thị dữ liệu trên component.

0.2. Công cụ và project baseline

Thành phần	Yêu cầu	Kiểm tra nhanh
JDK	21	java -version
Maven	Boot 3.x compatible	mvn -v
Node.js/npm	Bản tương thích Vite hiện tại	node -v / npm -v
Backend	Spring Boot API chạy ở port xác định, ví dụ 8080	GET bằng Postman
Frontend	React/Vite chạy ở port xác định, ví dụ 5173	Browser mở app
SQL Server	DB kết nối được	Startup log / SSMS
Browser DevTools	Console + Network	Quan sát fetch/XHR/OPTIONS
Git	Baseline sạch	git status / git log

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
React có thể gọi REST API; Spring Boot trả JSON; Axios trả Promise.	Vì sao Postman gọi được nhưng browser bị CORS?; Base URL nên đặt ở đâu?	Một click “Load Orchids” thực sự tạo ra những HTTP request nào?

0.4. Pre-flight checklist

- Backend compile/start không lỗi.
- GET API thành công bằng Postman trước khi tích hợp React.
- Frontend npm install và npm run dev thành công.
- Ghi rõ FE origin và BE origin thực tế.
- Mở DevTools Network trước khi test.
- Tạo commit/tag gợi ý: s20-before-fullstack-integration.

```
Backend:
  mvn -q -DskipTests compile
  mvn spring-boot:run
Frontend:
  npm install
  npm run dev
```

I. TÓM TẮT CHỦ ĐỀ & MỤC TIÊU BÀI HỌC

1.1. Slot 18/19 -> Slot 20: từ API độc lập đến browser-integrated system

Nền tảng đã có	Đã làm được	Câu hỏi Slot 20 trả lời
REST Controller + JPA	API chạy độc lập, test được bằng Postman	React gọi endpoint nào và URL được cấu hình ở đâu?
React/Vite	Component render dữ liệu local/mock	Dữ liệu thật đi từ Spring Boot vào state/UI như thế nào?
HTTP contract	Biết status/body	Browser còn áp dụng origin/CORS rule gì?
Lab/project guidance	Có FE và BE baseline	Làm sao tích hợp, debug và chứng minh end-to-end?
Deployment awareness	Biết dev ports	Staging/prod khác localhost ở API URL và CORS ra sao?

1.2. Mục tiêu học tập

- Giải thích SPA full-stack và vai trò riêng của React frontend, Spring Boot REST backend và database.
- Phân biệt URL, origin, same-origin policy và cross-origin request bằng ví dụ localhost:5173 <-> localhost:8080.
- Thiết kế API client phía React với base URL rõ ràng; tránh hard-code rải rác.
- Dùng Axios/fetch để gửi request và xử lý success/loading/error ở mức course-ready.
- Giải thích CORS, simple request, preflight OPTIONS và các header Access-Control-* ở mức quan sát được trên DevTools.
- Cấu hình CORS trong Spring MVC đúng scope cho môi trường development; hiểu @CrossOrigin và global WebMvcConfigurer.
- Trace một request từ component event/useEffect -> API client -> browser network -> Spring MVC -> service/repository -> DB -> JSON -> state/render.
- Chẩn đoán được lỗi phổ biến: wrong URL, ERR_CONNECTION_REFUSED, 404/405/415/500, CORS blocked, preflight failure.
- Phân biệt dev-time proxy, backend CORS và production reverse proxy/same-origin deployment; hiểu trade-off.
- Chuyển kiến thức sang project lớn với environment-specific API URL, least-privilege CORS và observability.

1.3. Mental model của Slot 20

```
User action / useEffect
-> React component
-> api/orchidClient.js
-> resolve BASE_URL + path
-> Axios/fetch creates browser request
-> Browser Same-Origin Policy check
  -> if cross-origin: CORS / maybe OPTIONS preflight
-> HTTP request to Spring Boot
-> DispatcherServlet -> Controller -> Service -> Repository -> SQL Server
<- JSON + status + headers
<- Browser CORS validation
<- Axios/fetch resolves/rejects
-> setState(...) / error state
-> React re-render
```

Hai lớp phải tách rõ: (1) Application/API correctness: endpoint, status, JSON, DB. (2) Browser access policy: CORS/origin. Postman có thể chứng minh lớp (1) nhưng không chứng minh lớp (2).

1.4. “FE gọi được BE” chưa đủ

Mức	Biểu hiện	Rủi ro
Demo beginner	Hard-code http://localhost:8080 trong nhiều component; @CrossOrigin("**") để “hết lỗi”.	Khó đổi môi trường; mở quá rộng; debug kém.
Course-ready	Central API client + env config; CORS đúng FE origin; trace bằng DevTools; error/loading rõ.	Đủ mục tiêu Slot 20.
Production-ready direction	Environment config, reverse proxy/gateway, least privilege CORS, auth/credentials policy, telemetry, retry/caching có chủ đích.	Cần deployment/security governance ngoài core lab.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG PHẢI NẮM VỮNG

2.1. Full-stack SPA architecture – boundary trước framework

Layer	Vai trò course-ready	Không nên trộn
React UI	Render, interaction, local UI state, gọi application API client	SQL/JPA/business rule server-side
API client	Base URL, path, common headers, request wrapper/error normalization	Component layout/UI concern
Browser	HTTP stack, cache, cookie policy, same-origin/CORS enforcement	Business rule
Spring Controller	HTTP contract, binding, status/body	UI state
Service/Repository	Use case + persistence	Browser/CORS-specific UI handling
SQL Server	Persistent state	HTTP contract

2.2. URL, origin và API URL

Origin = scheme + host + port. http://localhost:5173 và http://localhost:8080 khác origin vì port khác. Path không thuộc origin. Vì vậy /api/orchids và /api/orders trên cùng scheme/host/port vẫn same-origin.

Ví dụ	Origin	Kết luận
http://localhost:5173/app	http://localhost:5173	Frontend dev
http://localhost:8080/api/orchids	http://localhost:8080	Backend dev – cross-origin với FE
https://app.example.com	https://app.example.com	Production frontend
https://app.example.com/api/orchids	https://app.example.com	Same-origin nếu reverse proxy route /api tới backend

2.3. Base URL – đừng hard-code rải rác

```
// .env.development
VITE_API_BASE_URL=http://localhost:8080

// src/api/httpClient.js
import axios from "axios";

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10000,
  headers: { "Content-Type": "application/json" }
});
```

Production note: Vite expose biến có prefix VITE_ vào client bundle; không đặt password/secret/token server-side trong VITE_* env. Giá trị này là configuration public của frontend, không phải secret.

2.4. React -> API: request state phải có mô hình

```
const [orchids, setOrchids] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);

useEffect(() => {
  let active = true;
  setLoading(true);
  http.get("/orchids/")
    .then(res => { if (active) setOrchids(res.data); })
    .catch(err => { if (active) setError(err); })
    .finally(() => { if (active) setLoading(false); });
  return () => { active = false; };
}, []);
```

State	Ý nghĩa	UI evidence
loading	Request đang chạy	spinner/skeleton
success/data	Có response hợp lệ	list/detail
empty	200 nhưng danh sách rỗng	empty state
error	Network/CORS/HTTP failure	error message/retry phù hợp

2.5. Same-Origin Policy và CORS – hiểu đúng vấn đề

- Browser hạn chế script đọc response từ origin khác nếu server không cấp quyền phù hợp.
- CORS là cơ chế HTTP-header để server khai báo origin/method/header nào được phép.
- CORS không phải cơ chế authentication; cho phép origin không đồng nghĩa user được quyền truy cập business resource.
- Postman/cURL không thực thi browser same-origin policy theo cách browser làm, nên “Postman chạy” chưa đủ.
- CORS error thường phải debug ở server config + browser Network/Console, không chỉ trong code React.

2.6. Simple request vs preflight OPTIONS

Loại	Browser làm gì	Ví dụ
Simple/eligible request	Gửi actual request trực tiếp, sau đó kiểm tra CORS response headers	Một số GET/POST với safelisted headers/content-type
Preflighted request	Gửi OPTIONS trước để hỏi server có cho method/header dự kiến hay không	PUT/DELETE hoặc request có Authorization/custom headers thường dẫn tới preflight

```
OPTIONS /api/orchids/1
Origin: http://localhost:5173
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: content-type

HTTP/1.1 200/204
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,POST,PUT,DELETE
Access-Control-Allow-Headers: Content-Type
```

2.7. CORS trong Spring MVC: local vs global

```
// Controller-level: phù hợp demo nhỏ
@CrossOrigin(origins = "http://localhost:5173")
@RestController
@RequestMapping("/orchids")
public class OrchidController { ... }
```

```
// Global: dễ quản lý nhất quán hơn cho /api/**
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5173")
            .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
            .allowedHeaders("*")
            .maxAge(3600);
    }
}
```

Security rule: Nếu dùng credentialed CORS (cookie/HTTP auth), không dùng Access-Control-Allow-Origin: *; origin phải cụ thể và chính sách credentials phải được thiết kế cùng Spring Security/CSRF ở slot phù hợp.

2.8. Dev proxy – hữu ích nhưng không được làm sinh viên quên CORS

```
// vite.config.js - development only
export default defineConfig({
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:8080",
        changeOrigin: true
      }
    }
  }
});

// React gọi relative URL
http.get("/api/orchids");
```

Trade-off: Dev proxy giúp FE gọi relative URL và giảm friction trong development, nhưng có thể che khuất kiến thức CORS. Trong Slot 20 nên cho sinh viên quan sát CORS trực tiếp trước, sau đó mới so sánh proxy/reverse proxy.

2.9. Axios/fetch: abstraction vừa đủ

Concern	Nên centralize	Không nên làm ở mỗi component
Base URL	axios instance / API module	Ghép localhost string thủ công
Timeout	HTTP client config	Mỗi request một kiểu
Error normalization	Interceptors/wrapper ở mức phù hợp	Mỗi component parse error khác nhau
Auth header (future)	Interceptor/policy sau khi học security	Copy token logic khắp UI

2.10. Error taxonomy – network khác HTTP error

Nhóm	Ví dụ	Dấu hiệu
URL/network	Sai port, backend down, DNS/TLS	ERR_CONNECTION_REFUSED / Network Error; không có HTTP response hợp lệ
CORS	Origin/header/method không được cho phép	Console CORS; Network có OPTIONS hoặc response bị browser chặn
HTTP contract	404/405/415/400/500	Có status/response từ server
Application data	200 nhưng shape/field khác FE kỳ vọng	Render lỗi/undefined; contract mismatch

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC TRONG SẢN PHẨM

3.1. Case study xuyên suốt – Orchid Management / Lab 05

Phía	Thành phần	Trách nhiệm integration
Frontend	React/Vite – ListOfOrchids, EditOrchid, App	Gọi API, hiển thị loading/error/data, update UI state
Client boundary	Axios/API module	Base URL + endpoint + JSON + error mapping
Browser	localhost:5173	Same-origin/CORS enforcement, DevTools evidence
Backend	Spring Boot – OrchidController	Expose GET/POST/PUT/DELETE
Persistence	Spring Data JPA + SQL Server	Persist Orchid/Category

3.2. Từ UI use case -> API contract -> UI state

UI use case	HTTP	Success UI	Failure UI
Load orchids	GET /orchids/	Render list	Error banner/retry
View detail	GET /orchids/{id}	Detail/modal	404 -> not found
Create	POST /orchids/	Append/refetch + success	400 validation / 500
Update	PUT /orchids/{id}	Update state/refetch	404/400/conflict future
Delete	DELETE /orchids/{id}	Remove from list	404/500; keep UI consistent

3.3. Development, staging, production – cùng code, khác configuration

Environment	Frontend URL	API URL	CORS direction
Local dev	http://localhost:5173	http://localhost:8080	Explicit local origin hoặc dev proxy
Staging	https://staging-app.example.com	https://staging-api.example.com	Allow staging frontend origin
Production separate origins	https://app.example.com	https://api.example.com	Strict production origin + credentials policy nếu có
Production same-origin proxy	https://app.example.com	https://app.example.com/api	Browser same-origin; proxy/gateway route backend

3.4. Beginner design vs production direction

Vấn đề	Người mới thường làm	Production direction
API URL	Hard-code localhost	Environment-specific config / relative URL behind gateway
CORS	Allow * mọi nơi	Least privilege origins/methods/headers
Errors	console.log(error)	Normalized error model + user-safe UI
Request code	Axios gọi trực tiếp mọi component	API/service module; data layer rõ ràng
Secrets	Đặt key vào VITE_*	Không đưa secret server-side vào client bundle
Deployment	FE và BE như hai process local	CDN/static hosting + reverse proxy/API gateway/backend service
Observability	Chỉ nhìn console	Correlation/trace IDs, structured backend logs, browser network evidence

3.5. Current ecosystem update – awareness, không phá course baseline

Hệ sinh thái	Tình trạng 08/10/2026	Ý nghĩa cho SBA301
React	React docs latest 19.3 (v19.3.0 09/09/2026)	Mental model component/state/network vẫn giữ; không cần đưa feature mới vào core Slot 20 nếu không phục vụ integration.
Spring Boot	GA 4.1.1 (20/08/2026)	Course vẫn giữ Boot 3.x để nhất quán; production awareness cần biết Boot 4.x tồn tại.
Spring Boot 3.5.x	3.5.16 là OSS release cuối của line 3.5.x	Không nên dạy “3.x là mới nhất”; phải ghi rõ course baseline vs current ecosystem.
Vite	Env qua import.meta.env; VITE_* được bundle vào client; dev server hỗ trợ proxy	Dạy base URL/env + security of public config; proxy là extension.

Teaching decision: Slot 20 ưu tiên các invariants bền vững: HTTP/origin/CORS/API contract/configuration/debugging. Version-specific feature chỉ dùng để minh họa cập nhật công nghệ.

IV. HƯỚNG DẪN CHI TIẾT THỰC HIỆN VÀ TRIỂN KHAI

A. Demo baseline – cấu trúc project

```
demo_slot20/
├── backend/
│   ├── src/main/java/...
│   │   ├── controller/OrchidController.java
│   │   ├── service/...
│   │   ├── repository/...
│   │   └── config/WebConfig.java
└── frontend/
    ├── src/
    │   ├── api/httpClient.js
    │   ├── api/orchidApi.js
    │   ├── components/ListOfOrchids.jsx
    │   ├── App.jsx
    │   ├── .env.development
    │   └── vite.config.js
```

B. Bước 1 – Verify backend độc lập trước

- Start Spring Boot và xác nhận port.
- GET /orchids/ bằng Postman.
- Xác nhận status + JSON shape.
- Nếu bước này fail, chưa debug React/CORS.

C. Bước 2 – Xác định origin và integration contract

Item	Giá trị ví dụ	Evidence
FE origin	http://localhost:5173	Browser address bar
BE origin	http://localhost:8080	Startup log
Endpoint	/orchids/	Controller mapping/Postman
Full API URL	http://localhost:8080/orchids/	Network request URL
Expected JSON	List<Orchid> representation	Postman/browser preview

D. Bước 3 – Tạo frontend API configuration

```
# frontend/.env.development
VITE_API_BASE_URL=http://localhost:8080
// frontend/src/api/httpClient.js
import axios from "axios";

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10000,
});
```

```
// frontend/src/api/orchidApi.js
import { http } from './httpClient';
export const getOrchids = () => http.get("/orchids/");
export const getOrchid = id => http.get(`/orchids/${id}`);
export const createOrchid = body => http.post("/orchids/", body);
export const updateOrchid = (id, body) => http.put(`/orchids/${id}`, body);
export const deleteOrchid = id => http.delete(`/orchids/${id}`);
```

E. Bước 4 – Gọi GET từ React và trace network

```
// ListOfOrchids.jsx (skeleton)
useEffect(() => {
  setLoading(true);
  getOrchids()
    .then(({ data }) => setOrchids(data))
    .catch(setError)
    .finally(() => setLoading(false));
}, []);
```

Trace evidence: Trong DevTools Network, ghi lại Request URL, Method, Status, Origin request header, response CORS headers (nếu cross-origin), timing và response preview.

F. Bước 5 – Cố ý để CORS fail trước khi sửa

11. Tạm thời bỏ CORS config backend.
12. Reload FE và quan sát Console + Network.
13. So sánh với Postman vẫn có thể gọi API.
14. Viết một câu kết luận: server có chạy không? browser chặn ở boundary nào?

G. Bước 6 – Configure CORS có chủ đích

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
  @Override
  public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**") // course demo; production nên scope hẹp hơn
      .allowedOrigins("http://localhost:5173")
      .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
      .allowedHeaders("*")
      .maxAge(3600);
  }
}
```

Production correction: Không mặc định copy “/** + * headers” sang production. Nên scope theo /api/**, origin cụ thể, methods/headers thực sự cần và phối hợp Spring Security khi có credentials.

H. Bước 7 – Trace một GET end-to-end

```
React render
-> useEffect()
-> getOrchids()
-> axios instance resolves baseURL
-> browser GET http://localhost:8080/orchids/
-> CORS processing
-> DispatcherServlet
-> OrchidController.getAll(...)
-> OrchidService...
-> OrchidRepository.findAll()
-> Hibernate/JDBC -> SQL Server
<- JSON response + CORS headers
<- Axios response.data
-> setOrchids(data)
-> React re-render list
```

I. Bước 8 – Trace POST/PUT và preflight

```
User clicks Save
-> React builds JSON
-> PUT/POST request
-> browser may send OPTIONS preflight
-> Spring CORS validates Origin/Method/Headers
-> actual request (if allowed)
-> Controller -> Service -> Repository -> DB
<- status/body
-> UI update / error state
```

J. Bước 9 – Browser Network + backend log correlation

Quan sát	Frontend/Browser	Backend
Request identity	URL + method + timestamp	Access/application log
Status	Network status	Controller/exception output
CORS	OPTIONS + Access-Control-*	CORS config/mapping
Body	Request Payload / Response Preview	@RequestBody + service input
Latency	Timing tab	SQL/log duration nếu có

K. Bước 10 – Integration test matrix

Test	Request/action	Expected browser/UI	Backend/DB verify
T01	Load FE	App render, no network error	Backend independent
T02	GET orchids	200 + list	DB unchanged
T03	Wrong API port	Network error	Backend may be healthy on another port
T04	CORS disabled	Browser blocked	Postman may still succeed
T05	POST valid	201 + UI reflects new item	New row
T06	PUT valid	200 + updated UI	Row updated
T07	DELETE	204 + item removed	Row deleted
T08	Backend 404	Not-found UI/handled error	No unintended DB change
T09	Preflight failure	OPTIONS failure + actual request not sent	Check CORS config
T10	Staging env simulation	Correct base URL selected	No localhost hard-code

L. Bước 11 – Break-It / Diagnose workshop

Break	Triệu chứng	Chẩn đoán
Sai API port	ERR_CONNECTION_REFUSED / Network Error	Network Request URL + backend startup port
Sai path	404	Controller mapping + request URL
Bỏ CORS origin	CORS blocked	Console + response header
PUT không allow method	Preflight fail	OPTIONS + Access-Control-Allow-Methods
Authorization header future nhưng CORS không allow header	Preflight fail	Access-Control-Request-Headers / Allow-Headers
Backend down	No response	Network + process/log
JSON field mismatch	UI undefined/render problem	Response Preview + component mapping
Hard-code localhost khi build staging	Calls local machine	Built config/env review

M. Bước 12 – Production bridge: reverse proxy/same-origin

```
Browser https://app.example.com
-> GET /api/orchids
-> Nginx / API Gateway routes /api/* to Spring Boot service
-> backend response
<- same public origin from browser perspective
```

Điểm cần hiểu: Reverse proxy không “thay thế” kiến trúc CORS; nó là một deployment architecture có thể làm browser request trở thành same-origin. CORS vẫn cần khi frontend và API thực sự ở origin khác.

N. Slot 20 Integration Gate

Gate	Bảng chứng bắt buộc
G1 Baseline	FE + BE start; API Postman success
G2 API config	Base URL central, không hard-code rải rác
G3 Browser trace	Ít nhất 1 GET trace từ React -> DB -> React
G4 CORS understanding	Giải thích origin và quan sát failure/success

Gate	Bảng chứng bắt buộc
G5 Preflight	Quan sát/giải thích ít nhất 1 OPTIONS khi phù hợp
G6 UI state	Loading + success + error evidence
G7 Persistence	POST/PUT/DELETE verify DB hậu điều kiện
G8 Break/Fix	Ít nhất 2 lỗi integration được chẩn đoán
G9 Environment	Có dev/staging/prod URL strategy worksheet
G10 Explain	Human verification pass: không chỉ demo chạy

V. KINH NGHIỆM THỰC TIỄN, LỖI THƯỜNG GẶP VÀ DEBUGGING

Vấn đề	Triệu chứng	Cách xử lý
Backend chưa chạy	Network Error / refused	Kiểm tra process, port, health/API bằng Postman
Sai API base URL	404/refused/request sai host	Inspect final Request URL; centralize config
CORS missing origin	Browser blocked	Server add explicit allowed origin
Preflight 403/404	Actual PUT/DELETE không chạy	Cho OPTIONS qua CORS/security chain; inspect methods/headers
Wildcard + credentials	Browser reject/policy sai	Explicit origin + credential policy
415	Unsupported Media Type	Content-Type + controller consumes/body
500	Unhandled backend exception	Backend stack trace; đừng gọi mọi lỗi là CORS
200 nhưng UI không render	Data shape mismatch/state bug	Preview JSON + component mapping
useEffect loop	Request lặp	Dependency array/state update analysis
Duplicate request dev	Effect/lifecycle/tooling behavior	Phân biệt dev behavior với production; inspect reason before “fix”
Secrets trong VITE_*	Secret xuất hiện client bundle	Move secret to backend; VITE_* chỉ public config
CORS * trong production	Attack surface rộng	Least privilege origin/method/header policy

5.1. Debugging decision tree

```

FE không có dữ liệu
|- Network tab có request?
|   |- NO -> event/useEffect/API-client path
|   |- YES
|       |- Request URL đúng? NO -> base URL/path/env
|       |- Có HTTP response? NO -> backend down/network/TLS/DNS
|       |- CORS error? YES -> inspect Origin + OPTIONS + Allow-* headers
|       |- Status 4xx/5xx? -> HTTP/application contract
|       |- 2xx nhưng UI sai -> JSON shape -> state update -> render
|
Backend nhận request nhưng DB sai
-> service rule -> repository -> SQL/log -> DB state

```

5.2. Best practices ở mức Slot 20

- Luôn chứng minh backend độc lập trước khi debug frontend integration.
- Không hard-code API URL ở nhiều component.
- CORS config phải theo origin thật; không dùng wildcard như “thuốc chữa lỗi”.
- Browser Network tab là evidence chính cho integration bug.
- Phân biệt Network Error, CORS error, HTTP error và UI/state error.
- Không đưa secret server-side vào Vite environment variables.
- UI phải có loading/error/empty/success state tối thiểu.
- Production ưu tiên least privilege CORS và environment-specific configuration.
- Khi có reverse proxy, vẫn phải hiểu public origin và internal service routing.
- Mỗi lỗi integration quan trọng nên có regression test/checklist khi phù hợp.

VI. BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP

Bài	Yêu cầu	Evidence
1. Origin map	Liệt kê FE origin, BE origin, API URL, same/cross-origin	Worksheet
2. GET integration	React load Orchid list từ BE	Network + UI
3. Base URL refactor	Loại bỏ hard-code, dùng env/client module	Source diff
4. CORS failure	Tắt CORS, giải thích vì sao Postman khác browser	Console/Network note
5. CORS fix	Cho đúng local origin	Response header evidence
6. Preflight	Quan sát PUT/DELETE/headers tạo OPTIONS khi có	OPTIONS trace
7. Error state	Hiển thị backend down/404 hợp lý	UI evidence
8. DB post-condition	POST/PUT/DELETE verify DB	Before/after
9. Proxy comparison	So sánh direct CORS vs Vite dev proxy	Trade-off note
10. Production transfer	Thiết kế URL/CORS cho staging + production	Deployment worksheet

6.1. Bài tập tổng hợp – Complete demo_slot20 + chuẩn bị Lab 05

- Hoàn thiện full-stack Orchid Management với FE React và BE Spring Boot.
- Tạo API client module và environment-specific base URL.
- Ghi rõ FE/BE origin và chính sách CORS development.
- Trace một GET và một write operation end-to-end.
- Chạy test matrix positive + negative.
- Thực hiện ít nhất hai Break-It -> Diagnose -> Fix.
- Viết README: cách chạy FE/BE, ports, env, API URL, CORS, known limitations.
- Tạo “Production Migration Note”: staging/prod URL, reverse proxy option, CORS policy, secret handling.
- Áp dụng AI Self-Discovery Prompt 1 và lưu AI Verification Worksheet.

VII. FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa ngắn	Ví dụ Slot 20
SPA	Ứng dụng web cập nhật UI động mà không full-page reload cho mỗi tương tác	React app
API Base URL	Phần URL chung của API	http://localhost:8080
Origin	scheme + host + port	http://localhost:5173
Same-Origin Policy	Browser policy hạn chế script truy cập cross-origin	FE 5173 gọi BE 8080
CORS	HTTP-header mechanism cấp quyền cross-origin	Access-Control-Allow-Origin
Preflight	OPTIONS request hỏi quyền trước actual request	PUT/DELETE/custom headers
Origin header	Origin của request do browser gửi	Origin: http://localhost:5173
Access-Control-Allow-Origin	Server header cho biết origin được phép	http://localhost:5173
Access-Control-Allow-Methods	Methods server cho phép cross-origin	GET,POST,PUT,DELETE
Access-Control-Allow-Headers	Request headers được phép	Content-Type, Authorization
Axios instance	HTTP client được cấu hình dùng chung	baseURL + timeout
Vite env	Biến build-time/client config qua import.meta.env	VITE_API_BASE_URL
Dev proxy	Vite dev server forward path tới backend	/api -> localhost:8080
Reverse proxy	Public server/gateway route request tới backend service	app.example.com/api
Network Error	Không có HTTP response usable ở client	Backend down/refused/CORS abstraction
Integration trace	Chuỗi FE -> browser -> BE -> DB -> FE	GET orchids end-to-end

VIII. RESEARCH BASE VÀ AI-BASED LEARNING

8.1. Nguyên tắc dùng AI trong Slot 20

- AI phải dựa trên source code FE/BE và config thực tế; thiếu file/env/log thì phải nói thiếu bằng chứng.
- Mọi claim về request flow phải kiểm bằng Browser DevTools Network + backend log/debugger.
- Mọi claim về CORS phải phân biệt browser policy với server authentication/authorization.
- Mọi claim về framework/version phải kiểm bằng official docs/release notes và ghi ngày truy cập.
- Phân biệt Course Baseline (Spring Boot 3.x + JDK 21 + React/Vite) với Current Ecosystem Update.
- Không đưa secret/API credential vào prompt hoặc client-side env chỉ để “AI giúp debug”.

8.2. AI SELF-DISCOVERY – 3 PROMPT CHUẨN

PROMPT 1 – Khám phá full-stack request flow step-by-step

Tôi đang học SBA301 Slot 20 và có một project gồm React/Vite frontend và Spring Boot backend. Hãy phân tích đúng source tôi cung cấp, không tự bịa file/method. Chọn một API GET Orchid và một API PUT/POST. Trace từ component event/useEffect -> API client -> URL resolution -> browser request -> CORS/preflight (nếu có) -> Spring Controller -> Service -> Repository -> SQL Server -> response -> Axios/fetch -> state update -> re-render. Với mỗi bước ghi file/class/function, input/output và evidence tôi phải xem trong DevTools/log/debugger. Nếu không đủ source, chỉ rõ file còn thiếu.

PROMPT 2 – Chuyển từ localhost sang staging/production

Từ project React + Spring Boot hiện tại, hãy đóng vai trò Senior Full-Stack/Platform Engineer. Thiết kế ba môi trường development, staging, production: frontend URL, API URL, environment variables, CORS origins/methods/headers, reverse proxy/gateway option, TLS, logging/trace ID. So sánh direct cross-origin API với same-origin reverse proxy. Không đưa secret vào VITE_* và giải thích trade-off từng lựa chọn.

PROMPT 3 – Technology update có kiểm chứng

Hãy kiểm tra official React, Vite và Spring documentation/release notes tại ngày tôi chạy prompt. Phân loại: (A) kiến thức Slot 20 vẫn là invariant nên giữ, (B) thay đổi phiên bản/tooling cần cập nhật, (C) production feature chỉ nên biết. Với mỗi claim ghi version, nguồn official và ngày truy cập. Không dùng blog thứ cấp làm nguồn duy nhất.

8.3. Current official reference snapshot (08/10/2026)

Nguồn official	Thông tin dùng trong giáo trình
React – react.dev/versions	Latest documented version 19.3; v19.3.0 released 09/09/2026.
Spring Blog – Boot 4.1.1	Spring Boot 4.1.1 released 20/08/2026.
Spring Blog – Boot 3.5.16	3.5.16 là OSS release cuối của line 3.5.x; migration awareness sang 4.x.
Spring Framework Reference – CORS	Spring MVC built-in CORS; preflight handled via HandlerMapping; explicit config required for cross-origin.
MDN CORS Guide	Origin-based browser policy; preflight OPTIONS; credentialed requests require explicit origin, không wildcard.
Vite Env & Modes	import.meta.env; VITE_* exposed client-side; không chứa secret.
Vite Server Proxy	Dev server hỗ trợ server.proxy để forward path.

IX. KỊCH BẢN GIẢNG DẠY 135 PHÚT

Thời lượng	Hoạt động giảng viên	Hoạt động sinh viên	Evidence
0–15'	Check baseline + nêu mental model integration	Start FE/BE, ghi origin/port	Baseline checkpoint
15–35'	Giải thích SPA boundary, URL/origin/SOP/CORS	Vẽ origin map	Concept map
35–55'	Demo GET React -> Spring Boot	Trace Network + backend	GET execution trace
55–75'	Break CORS + phân tích preflight	Quan sát Console/OPTIONS	Failure note
75–95'	Cấu hình CORS + API client/env	Sửa code và verify	Source + headers
95–110'	POST/PUT + DB post-condition	Thực hiện write operation	UI + DB evidence
110–125'	Debug workshop theo nhóm	Fix 1 lỗi được giao	Diagnose note
125–135'	Transfer production + exit ticket	Nộp 3 câu trả lời	Exit ticket

9.1. Exit ticket

24. Vì sao Postman có thể gọi API nhưng React browser bị CORS?
25. Origin của `http://localhost:5173/app` là gì và khác `http://localhost:8080` ở đâu?
26. Nếu production dùng `https://app.example.com/api` qua reverse proxy, browser còn nhìn thấy backend internal host hay không?

X. TÀI LIỆU THAM CHIẾU VÀ TRACEABILITY

10.1. Tài liệu môn học được dùng làm baseline

- Chapter 14 – Create Single Page Application: SPA, SPA architecture, ReactJS frontend, Spring Boot REST API backend, implementation SPA with ReactJS and Spring Boot.
- Lab 05 – Create Single Page Application with Spring Boot RESTful API: Spring Boot 3 + SQL Server + Spring Data JPA; React client dùng Axios; CORS configuration; React/Vite chạy local.
- Slot 18 template – cấu trúc giáo trình: Prepare -> Understand -> Trace -> Implement -> Break -> Diagnose -> Verify -> Transfer; integration gate; glossary; AI-based learning.
- SBA301 Syllabus – Chapter 14 / Lab 05 nằm trong CLO4 và course sequence trước Chapter 15 CRUD full-stack.

10.2. Nguồn official cập nhật production (truy cập 08/10/2026)

- React Versions: <https://react.dev/versions>
- Spring Framework CORS: <https://docs.spring.io/spring-framework/reference/web/webmvc-cors.html>
- Spring Boot 4.1.1 release: <https://spring.io/blog/2026/08/20/spring-boot-4-1-1-available-now/>
- Spring Boot 3.5.16 release: <https://spring.io/blog/2026/06/25/spring-boot-3-5-16-available-now/>
- MDN CORS Guide: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
- Vite Env Variables and Modes: <https://vite.dev/guide/env-and-mode>
- Vite Server Proxy: <https://vite.dev/config/server-options>

Traceability note: Nội dung “course baseline” bám Chapter 14/Lab 05; các chi tiết về env strategy, dev proxy, least-privilege CORS, reverse proxy, current versions và migration awareness được đánh dấu là production extension/update.